

EVALUACIÓN	Obligatorio 2	GRUPO	TODOS	FECHA	14/5/2026
MATERIA	Diseño de Aplicaciones 1				
CARRERA	AN/ID				
CONDICIONES	- Puntaje máximo: 25 puntos - Puntaje mínimo: 7 puntos - Fecha de entrega: 17/06/26 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip, rar o pdf) Uso de material de apoyo y/o consulta <u>Inteligencia Artificial Generativa</u> - Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso. - Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. - Se debe indicar: a. la /las herramientas utilizadas y b. el contexto de uso: por ejemplo, generación de ideas, redacción inicial, análisis de datos, corrección de estilo, etc. - Todo contenido producido por IAG debe ser revisado y verificado; cualquier error presente será responsabilidad del estudiante. - Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de “alucinaciones”, los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso. - El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema. - La defensa es obligatoria y eliminatoria. - La defensa será presencial salvo en casos en que se habilite la defensa oral sincrónica en forma remota. Defensa Fecha de defensa: entre semana 15 y 18 del semestre <u>La defensa es obligatoria y eliminatoria. El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.</u> IMPORTANTE: 1) Inscribirse 2) Formar grupos de hasta 3 personas del mismo dictado, salvo excepciones (ver acuerdo de inicio) 3) Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: “RECORDATORIO”) Aquellos de ustedes que <u>presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos</u>, por favor contactarse con la Coordinadora de cursos o Coordinación Adjunta antes de las 20:00h del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.				

Obligatorio II de Diseño de Aplicaciones I

Para la segunda entrega se deben considerar los siguientes cambios o agregados a la letra de la primera entrega.

Requerimientos de Implementación

1. Requerimientos No Funcionales

1.1. Persistencia de los datos

En esta nueva versión, se le solicita que todos los datos del sistema sean persistidos en una base de datos. Así, cuando se ejecute la aplicación, se empezará con los datos cargados con el último estado guardado antes de cerrar la aplicación.

Los datos se deben ir guardando en la base de datos durante la operativa de la solución. A medida que el usuario realiza las acciones, se debe ir almacenando, y **no** con un botón que salve todo “en un bloque” o al cerrar la aplicación.

Toda la información del sistema debe persistirse en una base de datos Azure SQL Edge, corriendo en un contenedor de Docker. El diseño debe contemplar el modelado de una solución de persistencia adecuada para el problema, utilizando Entity Framework Core (Code First).

Se espera que, como parte de la entrega, se incluyan dos respaldos de la base de datos: uno vacío y otro con datos de prueba. Se recomienda entregar el script .sql para ambos respaldos.

Es condición necesaria para obtener el puntaje mínimo del obligatorio que al menos una entidad del sistema pueda ser persistida.

1.2. Requerimientos de Diseño

Se espera que el diseño priorice el cumplimiento de los siguientes principios de diseño, los cuales se deben analizar en el apartado correspondiente en la documentación (ver en documentación Explicación de los mecanismos generales):

- **GRASP** – Controlador en alguna de sus variantes, en las clases del dominio del cumplimiento de experto
- **GRASP** - bajo acoplamiento y alta cohesión entre las capas de servicios y el dominio.

-
- **Ley de Demeter** – en particular en el dominio y en la capa de servicios.
 - **SOLID - SRP y OCP** en por lo menos una clase del dominio.

2. Requerimientos Funcionales

Nota: **La totalidad y detalle de los requisitos serán relevados a partir de consultas en el foro correspondiente en aulas**, las que deben tener un título descriptivo de su contenido. Tener en cuenta al momento de planificar el trabajo que las respuestas en Aulas **pueden demorar hasta 48 horas y dicho foro de consultas cerrará el día 6/06/26 a las 21hs.** Para evitar complejidades innecesarias, se realizaron simplificaciones al dominio del problema real.

A continuación, se describen los requisitos esperados para la entrega del obligatorio:

2.1 Extensión de Entidades y Nuevos Atributos

Se deben enriquecer las entidades existentes del Obligatorio 1:

- **Equipo:** Agregar la Bandera del país (almacenada en formato base64).
- **Partido:** Se deben registrar incidencias adicionales, como cantidad de tarjetas amarillas y rojas recibidas por cada equipo. A futuro, se van a agregar nuevos tipos de incidencias, como cambios, intervenciones del VAR, etc.

2.2 Nuevo Rol de Usuario y Notificaciones

Se debe incorporar un nuevo rol de usuario para extender el sistema de permisos:

- **Rol Periodista:** Este usuario puede consultar el fixture, ver estadísticas de equipos y rankings, pero no puede editar partidos ni gestionar usuarios.

2.3 Sistema de Ranking Dinámico

A diferencia de la primera entrega, donde el ranking era estático, el sistema ahora debe recalcular el nivel de los equipos tras cada partido finalizado (manual o simulado).

- **Ecuación de Actualización:** El ranking se actualizará según la fórmula:

$$\text{Ranking}_{\text{nuevo}} = \text{Ranking}_{\text{actual}} + [K \times (\text{Resultado} - \text{Probabilidad})] \times \text{MultiplicadorFase}$$

Donde los parámetros son fijos:

- Resultado: 1 (Victoria), 0.5 (Empate), 0 (Derrota).
- K (Factor de desarrollo): Valor fijo de 30 (determina la sensibilidad del cambio).
- Probabilidad (): Representa la expectativa de ganar según la diferencia de nivel. Se calcula como:

$$P = \frac{1}{1 + 10^{\frac{Ranking_{oponente} - Ranking_{actual}}{1000}}}$$

- MultiplicadorFase:
 - Fase de Grupos: 1.0
 - Rondas Eliminatorias: 1.5

Reglas de negocio para la Implementación

- Límites (Clamping): El ranking resultante nunca puede ser menor a 300 ni mayor a 2500, respetando los rangos de la entidad Equipo definidos en el primer obligatorio.
- Redondeo Determinístico: El resultado final de la ecuación debe redondearse al entero más cercano. En caso de empate (.5), se debe redondear hacia el entero superior para garantizar que todos los grupos obtengan el mismo resultado bajo las mismas condiciones (determinismo)

Ejemplo de cálculo:

- Equipo A (Ranking: 1500) vs Equipo B (Ranking: 1200) en Fase de Grupos.
- Gana el Equipo B (el "débil").
- para Equipo B = $1 / (1 + 10^{((1500-1200)/1000)}) \approx 0.33$.
- para Equipo B = $[30 * (1 - 0.33)] * 1.0 = +20.1$.
- **Nuevo Ranking Equipo B: 1220.**

2.4 Sistema de Notificaciones

Cada vez que se cargue un resultado (manual o por simulación), el sistema debe generar una notificación automática para los usuarios con rol Periodista.

Las notificaciones deben poder ser marcadas como leídas y dejar de aparecer en la vista principal del periodista.

2.5 Exportación de auditorías:

El sistema debe dar la posibilidad de exportar el **Fixture Final** y el **Log de Auditoría** de un cierto rango de fechas en formatos CSV y XLSX. La investigación correspondiente de qué librerías usar y cómo implementarlas queda a cargo de los estudiantes.

Esta funcionalidad solo estará habilitada para los roles de **Administrador** y **Periodista**.

2.6. Auditoría y Trazabilidad

Se debe extender el log de auditoría del primer obligatorio para registrar:

- Cálculos de cambios de ranking (valor anterior y valor nuevo).
- Ejecución de simulaciones (incluyendo la semilla utilizada).
- Exportación de archivos (quién descargó, qué archivo y cuándo).
- Lectura de notificaciones por parte de los periodistas.
- Todos los registros deben usar el **timestamp ISO-8601**

Los logs deben estar asociados al sistema y no a un torneo en particular.

2.7 Motores de Simulación Intercambiables

La letra anterior pide una funcionalidad de simulación basada en el RankingActual y una semilla. Para esta nueva entrega, en lugar de un único algoritmo, el sistema debe permitir elegir entre diferentes "Motores de Simulación". Por ejemplo: un motor "Probabilístico" (basado en el ranking actual) y un motor "Aleatorio Puro" (basado solo en la semilla).

Al crear el fixture se debe elegir que algoritmo utilizar para simular. Se debe mostrar en pantalla el nombre del simulador utilizado.

3. Requerimientos asociados al desarrollo con IA Generativa

Como parte de este curso, se espera que los estudiantes comiencen a utilizar y evaluar el uso de agentes de IA que los asistan en el desarrollo de la aplicación. A continuación, se detallan los usos esperados de este tipo de herramientas.

3.1. Creación de nuevo Skills e instrucciones para el agente de IA

Se debe demostrar el uso de al menos dos Skills diferentes a las utilizadas en el obligatorio 1 y se debe crear una nueva skill que escriba el Readme.md en el cual incluya:

- Breve descripción de la solución.
- Tecnologías utilizadas
- Estructura de la solución
- Instrucciones para su ejecución

El repositorio debe incluir un archivo de instrucciones para el Agente de IA con las principales instrucciones que el equipo haya definido para que el mismo realice el trabajo como el equipo desea.

Importante: Algunos skills pueden modificar el código de forma no esperada o crear vulnerabilidades. Se recomienda probarlas en un proyecto de prueba antes de utilizarlas en el obligatorio y si se reúsan de otros Marketplace o repositorios verificar que el creador sea confiable.

Entrega del obligatorio y datos de prueba

Se debe entregar una aplicación que contemple toda la funcionalidad descrita en este documento. **Se espera que la aplicación se entregue con una base de datos con datos de prueba**, de manera de poder comenzar las pruebas sin tener que definir una cantidad de datos iniciales. Por este motivo, la fuente de datos de prueba no debe estar embebida en el código.

El equipo deberá considerar que los datos de prueba entregados deben ser completos y permita ver escenarios de uso no triviales. Se espera que la base de datos incluya **como mínimo**:

- 2 usuarios Administrador

- 1 torneo cargado con los datos cargados para simular el fixture y con la primera fase (de grupos) simulada.
- 2 usuarios Editor
- 2 usuarios Periodistas
- 5 estadios
- Al menos cuatro equipos deben tener su bandera.
- Las entradas de auditorías para los datos arriba generados.

Instalación

El costo de instalación de la aplicación debe de ser mínimo y **documentado** adecuadamente. Indicando: ubicación de los ejecutables y librerías requeridas, los *strings* de conexión a modificar, ubicación de los *scripts* de la base de datos, usuario inicial de la aplicación (si aplica) y cualquier otro cambio y/o artefacto requerido para que la aplicación ejecute adecuadamente. Recordar que la solución debe correr con .NET Core 8. Se sugiere verificar que el programa se ejecute correctamente en los laboratorios de la universidad, ya que la demo / defensa se tomará de forma presencial. Ver sección “Demo al cliente / Defensa”.

Mantenibilidad

Se mantienen los requerimientos de la primera entrega. La propia empresa eventualmente hará cambios sobre el sistema, por lo que se requiere un alto grado de mantenibilidad, flexibilidad, calidad, claridad del código y documentación adecuada. Por lo tanto, el desarrollo de todo el obligatorio debe cumplir:

- Estar versionado en un repositorio git, siguiendo la metodología gitflow.
- Haber sido escrito utilizando TDD (desarrollo guiado por pruebas) lo que involucra otras dos prácticas: escribir las pruebas primero (Test First Development) y refactoring. De esta forma se utilizan las pruebas unitarias para dirigir el diseño.
- Cumplir los lineamientos de Clean Code (capítulos 1 al 10, y el 12), utilizando las técnicas y metodologías ágiles presentadas para crear código limpio.

Pruebas unitarias

Se mantienen los requerimientos de la primera entrega. Se requiere escribir los casos de prueba automatizados con el framework de unit tests visto en el curso, documentando y justificando las pruebas realizadas. Para hacer pruebas F.I.R.S.T. usando BD, se recomienda utilizar: [EF Core In-Memory Database Provider](#) (la aplicación “productiva” debe correr con la base de datos corriendo en Docker como se menciona anteriormente en la letra).

Control de versiones

Se mantienen los requerimientos de la primera entrega. La gestión del código del obligatorio debe realizarse utilizando el mismo repositorio Git de GitHub (github.com) perteneciente a la organización de la cátedra [IngSoft-DA1](#)

Documentación:

La documentación debe entregarse en un único documento digital (máximo 15 páginas, sin contar anexos) en formato PDF. Se deben **actualizar todos los diagramas que correspondan**, y agregar los solicitados a continuación y aquellos que se consideren necesarios para mostrar los cambios realizados.

A continuación, **se detallan los aspectos que cambian respecto a la entrega anterior:**

Carátula del documento (se mantiene sin cambios)

Descripción general del trabajo y del sistema (se mantiene sin cambios)

Diagramas de paquetes (se mantiene sin cambios)

Diagramas de clases (se introducen cambios)

- Diagrama de clases del dominio de la aplicación. (se mantiene sin cambios)
- **Diagramas de secuencia y de clases que intervienen** (se introducen cambios)
Para las funcionalidades de generación del fixture de etapa de grupos (para uno de los motores definidos) y Gestión de Usuarios se debe realizar:
 - Diagramas de clases a nivel de servicio, mostrando la relación entre servicios, clases de dominio y acceso a datos. Si existen relaciones entre servicios, también deben incluirse para comprender su colaboración.
 - Diagramas de secuencia que muestren cómo las clases involucradas participan de la funcionalidad.
- No incluir diagramas de la interfaz de usuario, salvo que implementen lógica de negocio.
- No es necesario incluir en los diagramas de secuencia el detalle de los parámetros de los métodos.

Tabla de responsabilidades (se mantiene sin cambios)

Explicación de los mecanismos generales (se introducen cambios)

Se debe analizar el cumplimiento de los requerimientos de diseño planteados (ver 1.2). Se espera que detallen que partes del diseño y que mecanismos utilizaron para cumplir con estos requerimientos.

Análisis detallado de dependencias (se elimina)

Aplicación de TDD (se introducen cambios)

- Requisito de evidencia de la aplicación de TDD a nivel del repositorio **(se elimina)**.
- **Cobertura de pruebas unitarias** (se mantiene sin cambios)

Pruebas funcionales (se elimina y se sustituye por la demo)

Explicación del algoritmo (se elimina).

Dependencias externas (se mantiene sin cambios)

Uso de IA Generativa (se introducen cambios)

- Creación de una Skill e instrucciones para el agente de IA
 - Explicar el funcionamiento del Skill creado y su ruta en el repositorio.
 - Documentar las principales instrucciones que contiene el archivo de instrucciones e incluir su ruta en el repositorio.

Sobre los anexos:

Los anexos deben utilizarse únicamente para profundizar en aspectos específicos. El lector debe poder comprender el contenido principal sin necesidad de consultarlos.

Importante:

Se espera que el documento siga un orden lógico, integrando diagramas y explicaciones según corresponda. La entrega será evaluada como si estuviera dirigida a un cliente real, considerando claridad, prolijidad y profesionalismo.

Entrega por Gestión

Es de carácter obligatorio que la última versión de la documentación sea entregada vía Gestión. En caso de que se quieran adjuntar los diagramas que contiene el documento sueltos como imágenes, se deben subir por Gestión en archivo zip.

No se aceptará documentación que se incluya en el repositorio en github.

No se aceptará que a main se suba un archivo comprimido con la entrega final (ej: Zip, Rar, otros), el contenido de main deberá ser la integración de la rama develop a la main

Entrega en el repositorio de GitHub

La entrega en el repositorio del equipo en la organización de GitHub ([IngSoft-DA1](#)) debe estar compuesta por los siguientes elementos:

- ≠ Una carpeta con la aplicación compilada.
- ≠ Código fuente de la aplicación y de los tests (merge a main), incluyendo el proyecto que permita probar y ejecutar.
- ≠ Release y Tag en la rama main con un nombre adecuado (ver <https://docs.github.com/es/repositories/releasing-projects-on-github/managing-releases-in-a-repository>). La hora tope para realizar el merge a main y crear el Release / Tag será - sin excepción - las 21:00 del día fijado para la entrega.
- ≠ La documentación NO DEBE incluirse en el repositorio, la misma solo se puede entregar por [Gestión](#).

Rúbrica de evaluación:

Puntaje total: máximo 25 puntos.

	Evaluación de	Pts.	Criterios de corrección
Funcionalidad	Implementación de la funcionalidad pedida y calidad de la interfaz de usuario.	7	Excelente: Se implementa toda la funcionalidad, tiene buena usabilidad y no hay errores importantes de funcionamiento. Correcto: Falta algún punto no central de la funcionalidad, existen detalles no bloqueantes de funcionamiento o la aplicación no es fácil de usar. No suficiente: Faltan partes importantes (por alcance o dificultad) de la funcionalidad. Existe gran cantidad de errores o funcionalidades no usables.
Diseño y documentación	Cumplimiento con los ítems solicitados en el apartado documentación	8	Excelente: Se presenta un documento completo en función de lo que se pide, ordenado y prolijo, que explica la solución entregada en todos sus aspectos. Se utiliza la notación vista en clase, de manera formal y correcta, para presentar los aspectos importantes del diseño. Se intercalan diagramas y explicaciones, que permiten comprender el diseño de la solución, las decisiones tomadas y la motivación detrás de las mismas. Correcto: Se presenta un documento prolijo que describe el diseño de la solución. Se describen los aspectos más importantes, pero hay errores en el uso de la notación. Falta describir aspectos importantes solicitados El orden de la documentación no permite seguir un hilo

			descriptivo. No suficiente: Falta detallar parte importante de la solución. No se usa la notación vista en clase, o se usa en forma equivocada o incompleta en repetidas ocasiones. Se presenta la documentación como una sucesión de diagramas sin explicación, o se omiten completamente items solicitados.
Calidad del código y metodología de desarrollo	Uso de prácticas de control de versiones y configuración. Desarrollo guiado por las pruebas (TDD) y técnicas de refactorio de código Buenas prácticas de estilo y codificación y su impacto en la mantenibilidad (Clean Code) Correcto uso de las tecnologías Concordancia entre el código y el diseño documentado Implementación de acceso a la base de datos	7	Excelente: El código es prolijo, fácil de entender y cumple con los puntos cubiertos en el curso sobre Clean Code. Lo construido coincide con lo detallado en la documentación de diseño. El acceso a la base de datos está bien implementado y se evitan las operaciones con grandes volúmenes de datos en memoria. Se persisten, como mínimo, la cantidad de entidades solicitadas Correcto: El código es prolijo en general, aunque presenta detalles a mejorar. Lo construido se corresponde en términos generales con lo presentado en el documento. Hay pruebas unitarias, aunque no se evidencia el uso de TDD. Se persisten, como mínimo, la cantidad de entidades solicitadas No suficiente: El código es desprolijo o difícil de entender. No cumple en repetidos casos lo propuesto por Clean Code. No hay pruebas unitarias. No se persisten la cantidad de entidades solicitadas
Requerimientos de IA Generativa	Cumplimiento de los requerimientos solicitados (skills, e instrucciones agente).	3	Excelente: Se crea una skill que realiza completamente lo solicitado y se explica su propósito. Se evidencia el uso de la misma. Se detallan las instrucciones para el agente, las cuales se encuentran bien definidas e incluyen instrucciones valiosas para este tipo de

	Completitud de los requerimientos asociados al uso de IA solicitados. Documentación completa y correcta respecto a lo solicitado		herramientas y alineadas con los principales temas vistos en el curso. Se especifican las rutas para encontrar los archivos. Correcto: Se crea una skill que realiza parcialmente lo solicitado o la explicación de su propósito es confusa. Se detallan las instrucciones para el agente, las cuales se encuentran bien definidas e incluyen instrucciones mínimas para este tipo de herramientas y alineadas con temas vistos en el curso, pero de cuestionable relevancia. Se especifican las rutas para encontrar los archivos. Se especifican las rutas para encontrar los archivos. No suficiente: Se crea una skill pero no realiza lo solicitado o la explicación de su propósito es insuficiente. Se detallan las instrucciones para el agente, las cuales se encuentran poco definidas o incluyen instrucciones mínimas para este tipo de herramientas. Las mismas están poco alineadas con temas vistos en el curso o son de cuestionable relevancia. No se especifican correctamente las rutas para encontrar los archivos.
--	--	--	--

Demo al cliente / Defensa

La defensa del obligatorio intenta:

- Verificar autoría y trabajo equilibrado entre los integrantes del equipo

La demo al cliente del trabajo intenta:

- Evaluar el conocimiento general de los integrantes del grupo sobre la solución propuesta. Todos los integrantes deben conocer toda la solución.
- Verificar el aporte individual al trabajo por parte de cada uno de los integrantes del equipo y en función de los resultados, se podrán otorgar distintas notas a los integrantes del grupo.

Se espera que cada uno de los integrantes haya participado en el desarrollo de parte significativa del obligatorio.

El mecanismo de defensa consistirá en una demo del producto por parte de los integrantes del equipo y una sección de preguntas y respuestas para comprobar autoría y trabajo en equipo.

Aspectos importantes sobre la demo/defensa:

- Al momento de la defensa deben tener la **demo preparada**. Funciona como prueba de aceptación por parte del cliente, presentan su producto final al cliente mostrando consideraciones que tuvieron durante la implementación, manejo de errores, diseño de la solución, qué funciona y qué no, etc. Es decir, todas las funcionalidades que no son demostradas por parte del equipo se tomarán como **no implementadas**, lo que llevará a la pérdida de esos puntos.
- Se espera que la demo tenga una duración de aproximadamente **15 minutos**.
- Deben usar los **datos de prueba que se entregaron**, para que funcionalidades tales como reportes brinden resultados interesantes. Si no entregaron datos de prueba, deben crearlos para la demo y aclararlo en el momento.
- La demo **NO** es para arreglar y ver cómo lo hacen funcionar, sino para mostrar lo que entregaron y funciona. Por lo tanto, se **ejecuta desde el release** con lo que entregaron, y no se permite cambiar código para que funcionen estos casos.
- La **calidad de la demo** afecta el puntaje de la funcionalidad. Una demo con preparación pobre **implicará pérdida de puntos**.

- La **demo se realizará de la forma** que comunique el equipo docente, pudiendo ser presencial o virtual, como se comunicará oportunamente. La **no asistencia** sin aviso previo implica **pérdida de todos los puntos**.
- En caso de defensa virtual, todos los integrantes deben asistir con cámara encendida; **si no, la defensa no se tomará**.
- La solución debe estar instalada y funcionando al momento de comenzar la demo. Deben estar listos para la llamada con **al menos 10 minutos de anticipación**. Es decir, si a mi equipo le toca a las 18:30hs, a esta hora **comienza** la defensa. Por lo tanto, deben estar conectados y listos a las 18:20.
- Si algún grupo necesita cambiar el horario, debe coordinarse con algún otro grupo y notificarnos antes del día de la demo.
- Es importante que **prueben el software con tiempo y con la copia de lo que entregaron**, así no tienen problemas al momento de la demo.
- Todos los estudiantes del equipo deben poder **compartir su pantalla** si el docente así lo indica, por ejemplo, para responder preguntas de código.
- Todos los estudiantes deben tener **abierta la solución de su aplicación en Rider (NO para ejecutar**, sino para mostrar código cuándo se le solicite).
- Deben estar todos los integrantes del grupo en su horario; no se esperará por algún integrante que no esté sin aviso previo, en la hora indicada para la defensa.
- En caso de que el docente considere que algún integrante del equipo no respondió satisfactoriamente a la defensa de autoría, puede asignar distintas notas a cada integrante. La quita de puntos puede ser total.

NOTA: El incorrecto funcionamiento de la instalación puede significar la no corrección de la funcionalidad. En el caso de defensa en el laboratorio, durante la defensa cada grupo contará con 15 minutos para la instalación de la aplicación. Luego de transcurridos los mismos se restan puntos al trabajo.

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem “Evaluaciones” y la instancia de evaluación correspondiente, que figura bajo el título “Inscripto”.
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en “Agregar”. El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en “Crear equipo”.

Cualquier integrante podrá:

- **Modificar la integración del equipo.**
- **Subir el archivo de la entrega.**

5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en “Subir” para iniciar la entrega. Cada equipo (hasta 3 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.

6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de Cursos o Coordinación Adjunta **antes de las 20:00h** del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.